JavaScript Typed Arrays

Back to JS page

Table of Contents

Typed Array Basics

Typed arrays are array-like objects that provide a mechanism for reading and writing raw binary data in memory buffers.

Unlike regular arrays, typed arrays always hold the same type of data and have a fixed length.

Common typed array types:

Type	Description	Size
`Int8Array`	8-bit signed integer	1 byte
`Uint8Array`	8-bit unsigned integer	1 byte
`Uint8ClampedArray`	8-bit unsigned integer (clamped)	1 byte
`Int16Array`	16-bit signed integer	2 bytes
`Uint16Array`	16-bit unsigned integer	2 bytes
`Int32Array`	32-bit signed integer	4 bytes
`Uint32Array`	32-bit unsigned integer	4 bytes
`Float32Array`	32-bit floating point	4 bytes
`Float64Array`	64-bit floating point	8 bytes

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Typed Array Basics</h4>
<p id="demo"></p>

<script>
// Create a typed array with initial values
const numbers = new Int8Array([10, 20, 30, 40, 50]);

let text = "Int8Array values:<br>";
for (let num of numbers) {
  text += num + " ";
}
text += "<br><br>";
text += "Length: " + numbers.length + "<br>";
text += "BYTES_PER_ELEMENT: " + numbers.BYTES_PER_ELEMENT + "<br>";
text += "Constructor: " + numbers.constructor.name;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result View Example

8 Bit Integers

`Int8Array` holds 8-bit signed integers (-128 to 127). `Uint8Array` holds 8-bit unsigned integers (0 to 255):

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>8 Bit Integers</h4>
<p id="demo"></p>

<script>
// Int8Array - signed (-128 to 127)
const signed = new Int8Array([-10, 0, 50, 127, -128]);

// Uint8Array - unsigned (0 to 255)
const unsigned = new Uint8Array([0, 100, 200, 255]);

let text = "Int8Array (signed -128 to 127):<br>";
for (let num of signed) {
  text += num + " ";
}
text += "<br><br>";

text += "Uint8Array (unsigned 0 to 255):<br>";
for (let num of unsigned) {
  text += num + " ";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result View Example

16-Bit and 32-Bit Integers

`Int16Array` holds 16-bit signed integers. `Int32Array` holds 32-bit signed integers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>16-Bit and 32-Bit Integers</h4>
<p id="demo"></p>

<script>
// Int16Array - larger range
const int16 = new Int16Array([1000, 20000, 30000]);

// Int32Array - even larger range
const int32 = new Int32Array([100000, 2000000]);

let text = "Int16Array (16-bit):<br>";
for (let num of int16) {
  text += num + " ";
}
text += "<br>BYTES_PER_ELEMENT: " + int16.BYTES_PER_ELEMENT + "<br><br>";

text += "Int32Array (32-bit):<br>";
for (let num of int32) {
  text += num + " ";
}
text += "<br>BYTES_PER_ELEMENT: " + int32.BYTES_PER_ELEMENT;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result View Example

Floating Point Numbers

`Float32Array` and `Float64Array` hold floating point numbers:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Floating Point Numbers</h4>
<p id="demo"></p>

<script>
// Float32Array - 32-bit floating point
const float32 = new Float32Array([1.5, 3.14, 9.99, 100.05]);

// Float64Array - 64-bit floating point (double precision)
const float64 = new Float64Array([0.123456789, 123456.789]);

let text = "Float32Array (32-bit float):<br>";
for (let num of float32) {
  text += num + " ";
}
text += "<br><br>";

text += "Float64Array (64-bit double):<br>";
for (let num of float64) {
  text += num + " ";
}
text += "<br><br>";

text += "Float32 BYTES_PER_ELEMENT: " + float32.BYTES_PER_ELEMENT + "<br>";
text += "Float64 BYTES_PER_ELEMENT: " + float64.BYTES_PER_ELEMENT;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result View Example

Uint8Array vs Uint8ClampedArray

The difference between `Uint8Array` and `Uint8ClampedArray` is how they handle values outside the 0-255 range:

- `Uint8Array` : wraps around (modulo 256)
- `Uint8ClampedArray` : clamps to 0 or 255

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Typed Arrays</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Uint8Array vs Uint8ClampedArray</h4>
<p id="demo"></p>

<script>
// Uint8Array - wraps around (modulo 256)
const uint8 = new Uint8Array([-10, 0, 100, 255, 300]);

// Uint8ClampedArray - clamps to 0 or 255
const clamped = new Uint8ClampedArray([-10, 0, 100, 255, 300]);

let text = "Uint8Array (wraps around):<br>";
for (let num of uint8) {
  text += num + " ";
}
text += "<br>(-10 wraps to " + uint8[0] + ", 300 wraps to " + uint8[4] + ")";
text += "<br><br>";

text += "Uint8ClampedArray (clamps):<br>";
for (let num of clamped) {
  text += num + " ";
}
text += "<br>(-10 clamped to " + clamped[0] + ", 300 clamped to " + clamped[4] + ")";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result View Example

Document

Document in project

You can Download PDF file.

Reference

- W3Schools JavaScript Typed Arrays